

Judged Whole

One Gate for Untrusted Verification Infrastructure

ANONYMOUS AUTHOR(S)

Verification infrastructure written by a language model cannot be trusted by construction, and neither can a proof it emits. We describe a system in which that is not a problem to be solved but the premise of the design. Every executable — interpreters, translators between languages, searches for evidence, certificate checkers, the maps that carry evidence home — is generated text, and none of it is believed: each enters through one gate that runs it against its own shipped cases and its own shipped mutants, and each result is graded by *where along its route the evidence was judged*. A witness is judged by replaying it through the interpreter of the language the question lives in; a certificate by re-checking it with that language’s checker; a claim with nothing behind it is judged by nothing and says so. The grade is a distance — certified at gap zero, checked one or more translations away, claimed when no judgment ever ran — and what a result still rests on, its residual, is computed from the log by removing everything upstream of the last judgment. The trusted base is exactly the admitted judges; the kernel that runs the gate stands outside it and is proved where it is mathematics, falsified where it is operation, and doubled by a clean-room second implementation that must agree byte for byte.

We measure the design at one frozen commit, and we measure it against faults nobody shipped. Of 1,672 sampled source mutants of the generated translators, carry-backs, and searches, forced past the gate and played on questions whose answers are known, 16,720 plays produce 133 wrong answers — 127 graded claimed, 6 graded checked, *none* certified — and every wrong answer’s residual names the faulty component. Of 700 mutants of the seven judges, the gate refuses half; a differential probe of the survivors finds six real soundness holes, and revisions that add only test vectors, each expected value confirmed by an outside tool that never enters the system, close them. On four benchmarks pinned from HWMCC’24 and SV-COMP 2025 the system settles 45 of 74, 26 of 79, 47 of 80, and 12 of 55 questions, 49 universal answers resting on a home judge alone; the generated searches are not competitive with the solvers of those competitions, and the paper reports the negatives as results.

1 INTRODUCTION

Verification infrastructure is now being written by language models: interpreters for the formats benchmarks arrive in, translators between them, invariant generators, proof scripts, and the checkers that are supposed to catch the rest. Each such artifact is text produced by a process that cannot be inspected and that is wrong at an unknown rate, and the question every deployment faces is what a result computed by such artifacts is worth. Two answers are current. One verifies the artifact: repair the proof until a checker accepts it, or generate the code together with a specification and a proof [16, 39]. The other distrusts the artifact and checks its outputs: certifying algorithms [27], proof-carrying code [31], validated verification witnesses [8]. Both answers are about one artifact at a time. Neither says what happens when the checker is generated too, when the translation that carried the program to the checker is generated, when the map that carries the checker’s verdict back to the original program is generated, and when the search that produced the evidence was generated last week by the same process.

This paper describes a system built on the premise that every one of those pieces is untrusted, and judged whole — and built that way in fact, not as a thought experiment. It started from an empty repository with a specification and nothing else, and everything that has entered since — every interpreter, translator, search, checker, and carry-back — is Python written by a language model and admitted through one gate. No human wrote any of it, and no human vouches for it line by line; what stands in for that is the subject of the paper. The separation that organizes it is a single sentence:

Generation produces syntax; only interpretation produces truth.

A *language* is a syntax with an *interpreter* — the program that runs its programs and reports named observables — and, where the language declares *certificate forms*, a *checker* for each. Interpreters and checkers are the *judges*, and the trusted base of the system is exactly the set of admitted judges, a list the kernel prints. Every other executable is a *transport*: a *translator* from one language to another, a *carry-back* that brings evidence found at the target home to the source, or a *search* that looks for evidence — a *witness* or a *certificate* — within a budget. A transport is untrusted whatever it computes; its output is only ever as good as the judgment it survives. **Searches do not decide; they write. Judges decide.**

The consequence for a result is that its *grade* is geometry. A question lives at a language; a *route* carries it across *pairs* of languages to a search; whatever comes back is judged where it lands, and the *gap* of a result is the number of hops between the question and the last judgment its evidence passed. Gap zero is *certified*: a witness replayed by the interpreter of the language the question lives in, or a certificate discharged by that language’s checker. A positive gap is *checked*: the certificate was discharged one or more hops away, and the result still rests on the translators between. No judgment at all is *claimed*. The *residual* of a result — what it still rests on — is computed from the log as the lineages of the hops inside the gap plus the judge that ran; every judgment removes everything upstream of it, so at gap zero the search that found the evidence, and the route it travelled, are not in the residual at all ([Theorem 4.2](#), proved in Lean).

A design like this makes two claims that can be wrong. The first is about transports: a faulty translator, carry-back, or search can cost answers but can never forge a *certified* one, and whatever it does corrupt says so in its grade and names it in its residual. The second is about judges: the trusted base is small enough, and tested well enough, to be believed. We test both by fault injection ([Section 6.1](#)). For the first we mutate every transport in the registry, *remove the gate*, force each mutant in, and play it on questions whose answer is known: 16,720 plays of 1,672 mutants yield 133 wrong answers, 127 of them *claimed*, 6 *checked* at gap one, and none *certified*; all 133 carry the faulty entry’s lineage in their residual, and seven of the mutants that answered wrongly had passed the gate. For the second we mutate the judges and let the gate, and everything admitted downstream of a judge, try to refuse them: half of 700 are refused, the survivors concentrate in the certificate checkers, and a differential probe finds six of them laxer than the checker they mutate — real holes in the trusted base. We close the holes without touching a judge, by revisions that add test vectors whose expected values are confirmed by tools that never enter the system: a model checker, a compiler, and the formal model of an instruction set.

The system’s own boards are the third measurement ([Section 6.3](#)). Two domains have entered: hardware, whose root language is BTOR2 [33], and software, whose root is a fragment of C; RISC-V is a third language, anchored by no domain and corroborated by the pairs on either side of it. Four generated pairs connect the three languages, six generated searches live at BTOR2 and C, and four benchmarks pinned from HWMCC’24 and SV-COMP 2025 stand at 45 of 74, 26 of 79, 47 of 80, and 12 of 55 questions settled, with 49 universal answers *certified* at gap zero and no contradiction. The generated searches are not competitive with the solvers whose verdicts sit beside the boards, a translation of hardware into C returned a partial on every route it opened, and ten times the clock bought one question; the paper reports these because the system cannot say otherwise.

Provenance. The premise applies to this paper as well. All of the system — the kernel, its tests, the second lineage of its pure half, the Lean development, every entry in the registry — and the instruments of [Section 6](#) — the fault-injection harness, the generators of test-vector expectations — are code generated by language-model agents (Anthropic’s Claude Fable, through the Claude Code command-line agent). Most of this paper’s text, its tables, and its figures are generated by the same agents, from the specification and the measurement records. No human reviewed the code or

the text for semantic correctness. The human contribution is the architecture: the separation of judges from transports, the gate, the decision that grades are computed and never asserted, and the choices of what to build, what to measure, and what to claim. What the reader is offered in place of trust in the writers is what the system offers in place of trust in its transports. No number here originates with a model: each is read from logs and stamps the kernel wrote, the boards regenerate from those logs byte for byte, the fault-injection counts are recomputed from the deposited records by one command, the proofs are checked by Lean and their axiom audit is printed at every build, and the test vectors' expected values were confirmed by tools that are not generative. The reader need not trust the writers, human or otherwise, beyond what the thesis itself requires.

Contributions.

- A design in which every executable of a verification system is untrusted generated text and one gate admits all of them: two kinds (language, transport), three judgments an artifact faces on crossing a pair (the square, replay, discharge), and two crossings nothing judges (Section 3).
- Grades as geometry: gap, grade, and residual computed from the log, with the order on results and the residual's properties proved (Sections 3.4 and 4).
- A kernel outside the gate made solid four ways — proof, tests, a clean-room second lineage, content pins — each worded no stronger than what it verifies (Section 4).
- A fault-injection method for such systems, and its results: the no-forge claim tested with the gate removed, the trusted base's exposure measured and localized, six holes found and closed by vector-only revisions with outside ground truth (Section 6.1).
- Measurements at one frozen commit, every number tied to a file in the tree, the negatives reported as results (Section 6.3).

What is not claimed. The square is checked empirically, per program, on a pair's corpus; per-program translation correctness is not proved. The seal around generated executables is a blank PATH and an emptied environment, measured by tests, and not claimed to be a proof. Half the sampled judge mutants survive admission; for most of them we can say only that no certificate we could find distinguishes them from the judge. The *corroborated* flag has fired only between two searches, never between two judges. And the boards are not compared to any solver's on a shared clock; the competitions' verdicts are recorded beside each question as testimony, never ranked.

2 ONE QUESTION, TWO ROUTES

Everything the kernel does can be told as the story of one question. Suppose a benchmark pins a C program p — a loop over values read from the outside world, an assertion inside — and asks: can the assertion fail within twenty steps? In the system that sentence is already formal. C is a *language*: a deterministic syntax plus a generated, deterministic interpreter I_C exposing named *observables* — here bad, whether an assertion failed, and depth, how many statements have run — and the certificate forms it can judge, each with a generated checker. The *question* is p , at C , with its *ask*: the observable bad, the mode exists, the bound 20. It lives at C , and every grade it will ever receive is a distance from that home. The benchmark enters as a *domain*: a *root* language, the one its programs are written in, together with its *anchors* — labels, supplied vectors, the recorded testimony of outside tools — the ungenerable half, and all that entering a new domain costs.

Route one: C to BTOR2. No search reasons about C directly; the model-checking searches live at BTOR2, a language of bit-vector machines. What connects the two is a *pair*: one correspondence, written down as generated code — a translator T that turns the elaborated control flow of p into a machine, one statement one transition, bad a predicate on machine state, and *carry-backs* Δ for

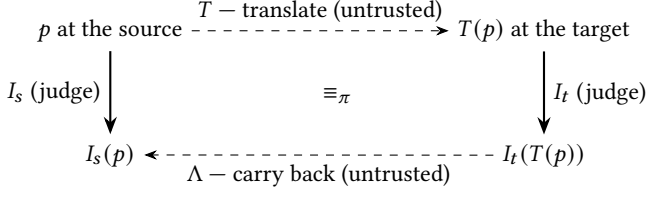


Fig. 1. The square, the judgment on programs. Dashed arrows are transports, generated and untrusted; solid arrows are the two interpreters, the judges. The square is closed empirically, per corpus program, at admission; the observable renaming π is the projection inside it. Where the translation turns one source frame into many target frames, a stimulus map and a bound map ride along and are judged inside the same square against the depths both interpreters report.

whatever will need to come home. Three kinds of artifact cross a pair, and each is judged on arrival by an interpreter run. The program crosses forward, and its judgment is *the square* (Fig. 1), closed for every program of the pair’s *corpus* — the programs it ships as its own controls — by running both interpreters: $I_s(p) \equiv_{\pi} \Lambda(I_t(T(p)))$ on the kept observables π . The horizontal arrows are untrusted syntax; the vertical arrows are the judges; nothing horizontal is believed until something vertical has run. A wrong translator — buggy, lazy, adversarial — does not produce wrong results; it loses squares and is refused at the gate.

At the stop. The route ends where evidence is written: a *search*, the one partial transport — budgeted, allowed to return an honest *partial* (how far it got, where it failed), and believed by no one. The searches at BTOR2 are generated solvers: explicit-state enumeration, bounded model checking, k-induction, and two property-directed searches of separate descent. Three kinds of *evidence* can come back down the route, and each is judged where it lands. A *witness* — the failing input sequence — is carried back, Λ renaming machine stimuli to the input sites of p , and judged by *replay*: I_C runs p on it where the question lives. The gap is zero, and gap zero is *certified*. No generated code can corrupt it: a wrong search, carry-back, or translator all fail the same way, by not producing a stimulus that replays. Witnesses need no declared form, because the judge is the interpreter itself, so a fresh domain certifies existential questions on the day its root interpreter is admitted. A *bare claim* — “no failure within bound k ” with nothing behind it — comes back unjudged, which is exactly why it cannot raise trust; its grade floors at *claimed*. A *certificate* — a k-induction kernel, a clause invariant — comes back and is judged by *discharge*: the checker of the language where it lands re-checks it. Discharged at BTOR2, the claim grades *checked* at gap one, and its *residual* is the lineage of the one hop it has not cleared plus BTOR2’s judge, the search no longer among them. Discharged at C — once the pair learns to carry the certificate itself home — the gap closes and the claim is *certified*. Each judgment removes everything upstream of it from the residual. If a replayed witness ever stands beside a covering universal claim, that is a recorded *contradiction*, never silently resolved: the witness stands, and the universal’s entire chain is marked.

Route two: C to RISC-V to BTOR2. The same question also travels a longer way: compiled to RISC-V — a language whose interpreter is execution of machine code, so reasoning can happen on the executable when that is the faster place — then encoded to BTOR2. Pairs compose into a route; its forward contract is the componentwise meet — fragments intersect, kept observables intersect, bounds cross each hop’s bound map — so a compiler pair that keeps only bad makes the whole route a bad-only route. What the longer route buys is a different *lineage*: its generated descent can share nothing with route one’s, and results of disjoint descent that agree earn the *corroborated* flag;

RISC-V itself, squeezed by squares against anchored neighbours on both sides, is corroborated from both directions at once. A *result* is one play of a route on one question, appended to the run's log with value, grade, gap, residual, and cost; cost is recorded and never ranked.

The loop. The player — a language model, or a human at the same command line — plays every question along every admitted route, reads the *frontier* (the questions no result has settled, each with how far every route got), conjectures the next entry, generates it, passes it through the gate, and plays again, until someone pulls the plug — safe at any moment, because the log is append-only and the best result per question only ever improves. The rest of this paper makes each word above exact (Section 3), says how the kernel that does this is itself made trustworthy (Section 4), lists what has been admitted (Section 5), and reports what fault injection and the plays found (Section 6).

3 THE DESIGN

The specification is written in a vocabulary of fifty words, listed in its last section, and this section uses no others. The registry holds exactly two primitive kinds; everything else — pairs, evidence, grades, routes — is derived from them, and the kernel computes with the derived forms without storing them as kinds.

3.1 Languages, judges, transports, domains

A **language** is a deterministic syntax plus a deterministic interpreter exposing named observables, plus the certificate forms the language can judge, each shipped with a generated checker. Interpreters and checkers are the **judges** — the system's only semantic devices — and the **trusted base** is exactly the set of admitted judges: a list the kernel prints, each judge with the anchors and controls that corroborate it. A *root* is a language benchmarks arrive in; its interpreter is the trusted floor, corroborated by the domain's anchors and by nothing generated. Every other language carries no parent taxonomy. Its interpreter is corroborated empirically by the squares of the pairs that connect it, and a language between two anchored neighbours, one square on each side against a different anchored interpreter, is corroborated from both sides at once, which no single pair can give.

A **transport** is a generated function on syntax, declaring its *fragment* — the programs it accepts; outside it, a transport refuses rather than guesses. There are three: the *translator* *T*, a source program to a target program; the *carry-backs* Λ , a target-side artifact to its source-side form — a witness input, a certificate, an observable name; and the *search*, the one partial transport, given a budget and deterministic for that budget, allowed to return *partial*. A route ends at exactly one search. Every transport is untrusted, whatever it computes.

A **domain** is a root language together with its *anchors* — the two things the loop cannot generate for itself: the format questions arrive in, and the ground truth (benchmark labels, supplied vectors, the recorded testimony of outside *oracles*) that corroborates the root's interpreter. A benchmark lives in exactly one domain, and a domain owns nothing else and fences nothing: every admitted language, pair, and search serves every domain, and the gate, not topical relatedness, is the only membership test.

3.2 Pairs: the three judgments, and two crossings nothing judges

A **pair** is the transports that share one correspondence between a source and a target language — its translator and its carry-backs — declared with a *direction* (exact, over, under), the observables it *keeps*, the artifacts it carries, measured cost, its lineage, and its *corpus*, the programs it ships as its own controls. The grouping is not redundancy: a witness carried back by one translator's carry-back and replayed against another translator's output is nonsense; the pair is the statement

that its transports are meaningful together. Three kinds of artifact move across a pair, and each is judged on arrival by an interpreter run; there is no other kind of check in the system:

artifact	moves	transport	judged on arrival by
a program	forward	T.py	the square : both interpreters agree on what is kept
a witness	back	lam_wit.py	replay : the source interpreter runs the program on it
a certificate	back	lam_cert.py	discharge : the source language's checker re-checks it

Two more things cross a pair, and neither is judged. A **bare claim** — “no failure within bound k ” with no certificate behind it — crosses back needing no transport, only the pair’s observable renaming and its *bound cap*; nothing judges it, and that is exactly why it cannot raise trust. A **hint** — seeds, candidates — crosses forward into a search, trust-inert by construction: whatever it seeds is judged on arrival, so a hint can move cost and never a grade.

The square is checked empirically, per corpus program, both interpreters run at admission; it is not a primitive and not a proof, and that remains the platform’s founding economy. Direction declares what may cross back — witnesses along exact and under pairs, universal objects along exact and over — and the square is the empirical evidence for the direction claim. A pair that reifies an unrolling declares a bound cap: an unbounded object crossing it arrives as a bound- k fact.

Frames that do not align. A pair whose translation changes the frame granularity — one source frame becoming many target frames, as one machine step becomes a run of statements — ships two more transports, judged inside the square. A *stimulus map* (lam_in.py) carries a source stimulus forward to the one the target interpreter runs on; the square runs the target side on its output, and the witness carry-back must be its inverse on the corpus. A *bound map* (lam_bound.py) carries an ask forward and a claim back: forward, the target depth at which a source firing at frame t lands; back, the largest source frame whose image the claim covers, or nothing. Neither is trusted: the gate checks the bound map’s structure — forward strictly monotone with infinity fixed, back its lower adjoint — and holds both maps to the depths the two interpreters report on every corpus stimulus; their mutants must fail there like any other transport’s. The player carries bounds through every hop, books a claim that ends below the first source frame as a partial, and never plays a route that revisits a language, so that a pair in each direction between two languages never plays the round trip.

Certificate forms belong to languages, not to searches. For a certificate to come home, what it looks like must be declared where it lands: a form named at a language and shipped with a generated checker. A pair’s certificate carry-back then maps form to form, per program, through the correspondence its translator already computes.

3.3 Evidence: what a search writes

A search is a transport from a language to evidence about its programs. It declares which observables it can target, it writes, and it pronounces nothing. A **witness** is an input; its judge is the language’s own interpreter, by replay, so no generated code stands between the payload and the trust event. A **certificate** is an object of a form the language declares — a k -induction kernel, a clause invariant, a box per program point — shipped with a checker under the language entry and admitted like everything else: determinism, example certificates that must discharge, mutant certificates that must not. Checkers are judges, part of the trusted base, and deliberately the simplest code in the system; smallness of judges is the honesty metric. A **bare claim** is legal and floors at the lowest grade — intended selection pressure: a search that writes checkable certificates outgrades one that writes bare claims at the same cost, so the ladder itself breeds certificate printers. A **partial** — how far the search got, where it failed — is what a route returns when nothing above came home. A

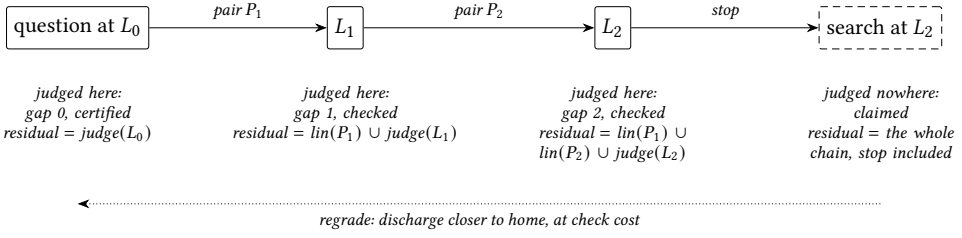


Fig. 2. Grades as geometry. A route of two pairs ends at a search; whatever the search writes is judged where it lands, and the gap counts the hops between the question and the last judgment. The residual is the lineage union over the gap segment plus the judge; every judgment removes everything upstream of it, so at gap 0 nothing but the home judge remains. A bare claim is judged nowhere and rests on the whole chain. Regrade moves a stored certificate down this ladder at check cost.

result that decides its question is *settled*: a replayed witness, or a universal claim covering the asked bound. A further question mode — liveness, resource bounds — is a new certificate form with a new checker, not a kernel change.

3.4 Grades are geometry

Trust is computed, per result, from where its evidence was judged (Fig. 2):

gap = the hops between the question and the last judgment the evidence passed
grade = *certified* if the gap is 0; *checked* if it is positive; *claimed* if no judgment ever ran
residual = the lineages of the hops still between the question and that judgment, plus the judge that ran
corroborated = results of disjoint lineage agree — an orthogonal flag

Each judgment removes everything upstream of it from the residual. A certificate discharged at the search's language removes the search from what the result rests on — the search may be garbage; the checker validated the object. Carried one hop back and discharged again, it removes that hop too. Discharged where the question lives, it removes the route entirely: *certified* is route-independence (Theorem 4.2). The fail-safe direction is definitional and universal: transport is untrusted, so a wrong or adversarial carry-back can only lose a grade, never forge one. Witnesses have one property worth stating once: the kernel always replays them where the question lives, chaining carry-backs across the route, so a witness is certified or it is not a result at all; an unplayable witness is evidence inside a partial, never an answer. A replayed witness is ground truth, and a covering universal claim beside it is a recorded **contradiction**: the witness stands, and the universal's entire chain — evidence, judgments, every transport it crossed — is marked falsified.

Lineage is a declaration about the descent of generated procedures. Two judges grown from the same ancestor never corroborate each other; the corroborated flag requires evidence whose descent sets are disjoint.

3.5 Routes, results, the order, the frontier

A **question** is a program, the language it lives in, and its *ask*: an observable, a mode, and a bound. A **benchmark** is a pinned finite set of questions with recorded provenance — a sha256 per program, labels where they exist. A **route** is a sequence of pairs ending at one search; its forward contract is the componentwise meet — fragments intersect, kept observables intersect, bounds cross each hop's bound map and bound caps take the minimum, costs add — and how far an artifact travels back is per kind: as far as every pair on the way carries it, its grade computed from where it was

last judged, not from what the route promises. A **result** is one play of a route on one question, appended to the log with everything the board needs:

$$result = \{ question, route, budget : \{ caps, spent \}, value, grade, gap, residual, lineage \}$$

where the value is a replayed witness, a universal claim $all(bound : k \mid \infty, cert?)$ graded by its gap, or a partial with a small typed core the kernel orders on — the bound reached — and free-form progress for the player to read.

Results order per question by one key, stated in the specification exactly because the mechanization and the second lineage of the kernel are both held to it: $(level, bound, grade, gap)$, compared lexicographically, strictly greater is better. Level: 0 a partial, 1 a universal claim below the asked bound, 2 settled. Bound: for a universal claim its bound, ∞ above every number; for a witness, above ∞ itself — a replayed witness stands above any universal claim on its question; for a partial the bound it reached. Grade: ungraded below *claimed* below *checked* below *certified*. Gap: a smaller gap is better, and “no check ever ran” sits below every finite gap. The incumbent survives only while it is strictly better; among records of equal key the latest wins, so the board shows the most recent adjudication of an equally good result. Cost is recorded in every result and never ranked.

Two performance moves are trust-free because they are judged. **Regrade** re-discharges a stored certificate closer to home: it costs check time, not search time, a shrinking gap is a strictly improving result under the order, and a revised judge re-derives a stored certificate’s residual the same way, so a proof is always read under the judges the registry holds now. **Hints** are forward artifacts that can move minutes of search and not one grade.

The **frontier** of a benchmark is the set of questions whose best result is not settled, each carrying that result and its progress. The registry and the log are append-only; best-per-question over an append-only log is monotone: the *ratchet* is a property of the data structure. Expanding the frontier means strictly improving some question’s best result — level, bound, grade, or gap; never cost. The frontier is drawn, not only listed: the *board* (one row per question, its best result with value, grade, gap, residual, route, and cost) and the *graph*, both pure functions of the log, both regenerating byte-identically.

The ledger. Cost says what a play spent; the ledger says what it bought, in bits — profiling, never a grade: the surprisal of a witness under the uniform measure the interpreter’s havoc rule fixes, the log-size of the stimulus space a bound- k claim exhausts, the compressed length of a certificate, and per pair an exchange rate measured on its corpus at admission. A search may ship a trust-inert ledger.py; what it writes is recorded beside every result.

3.6 The generation rule: oracles, not organs

Every implementation is generated, in Python, and lives as committed source inside its registry entry. No existing tools: no wrapped engines, no shelling out, no vendored binaries of someone else’s reasoning. The substrate is the Python interpreter the kernel runs on and a declared compiler where an accelerator needs building; nothing that reasons enters except as generated text through the gate. The rule bounds the system, not the evidence about it. At admission time a human or the generator may consult an **oracle** — an existing compiler or solver run entirely outside the sealed runner — and what the oracle says enters the registry the only way ground truth ever enters: as anchors with recorded provenance, corroborating a judge or a transport the way a benchmark label corroborates a root interpreter. An oracle never enters the trusted base, never runs inside a play, and its disagreement with a judge is a dispute to record and adjudicate, never a verdict. The per-question verdicts a competition archive publishes are anchors of this kind, and so are the

answers of the model checker, the compiler, and the model of an instruction set that confirmed the test vectors of [Section 6.2](#).

Enforcement is layered, and worded no stronger than what each layer verifies. Statically, there is no manifest field for pointing at a tool. Dynamically, the runner is sealed: every registered executable runs in its own process with an environment emptied to a blank PATH — blank rather than absent, because an unset PATH is replaced by the platform’s default search path, where a tool may sit — in a temporary working directory, under a wall cap, and every check runs twice with byte-compared output. Socially, every implementation is committed source. The seal makes reaching for a tool loud and the registry makes it visible; neither is claimed to be a proof.

Accelerators. Syntax may accelerate; semantics never does. An entry may ship one accelerator — the same implementation generated again in a faster language — only for the play-time transports, *T* and the search, whose outputs face judges downstream, and it is admitted solely by byte-agreement with its Python reference on every admission invocation. Judges always run the reference. The architecture maximizes the surface where the generator may be wrong for free — a wrong translator loses squares, a wrong search loses discharge, a wrong carry-back loses replay, none can forge — and the only things needing care are the judges.

3.7 The gate

Nothing — no language, pair, search, domain, or any of their implementations, accelerators included — enters the system except by admission, and admission evidence is stamped into the entry by the gate, never self-reported. One discipline for every kind: determinism measured (run twice, byte-compare), the checkable relation run on the entry’s own corpus, and two-sided **controls** — the intact implementation must pass and every supplied **mutant** must fail, because a checker that cannot be made to fail is unfalsifiable. Per kind: a language’s vectors — programs with pinned observables — interpret as expected, and each shipped checker discharges its example certificates and refuses its mutant ones; a pair’s every carried artifact round-trips per corpus program — squares close, stimuli replay, certificates discharge — with mutants per artifact kind; a search’s written evidence on its corpus must judge valid, its negative controls must fail, and its budget determinism must hold; a domain names its root and every anchor’s provenance. An entry that fails leaves no stamp and no trace in routing.

Revision, not mutation. An admitted entry is never edited. Every **stamp** pins the entry’s bytes with a content hash the loader re-verifies, defined in the specification rather than left to an implementation: over every regular file under the entry, excluding the top-level manifest, dotfiles, and bytecode, the map from entry-relative path to the sha256 of the file’s bytes, serialized as JSON with sorted keys, hashed once more. Extension arrives as a new entry <name>@<r> carrying a revision number and its predecessor’s hash. The gate for a revision is the ordinary kind gate plus conservativity: the new implementation must byte-agree with its predecessor on the predecessor’s whole checkable surface — its vectors or corpus, and for a language the corpora of every admitted pair bound to it. A name binds to its highest admitted revision; predecessors stay in the tree; results name the revision they ran. Adding a carry-back to an admitted pair is the intended common case: the old transports are the conserved surface, the new one is gated fresh.

3.8 The loop, the conjecture order, bootstrap from empty

The player is presented a benchmark and runs until a human pulls the plug: play — for every question, run the admitted routes within budget, the kernel recording results; read the frontier — the open questions with their progress, and the grades, since a board full of *claimed* and *checked* is itself frontier; conjecture, in the **conjecture order** — (a) new judging and searching for existing

languages, (b) new transports, including carry-backs retrofitted onto admitted pairs by revision and a translator from a root that reaches no hub to the nearest one, (c) new languages, proposed only when a translation or solving move keeps winning ad hoc; generate, gate, register; re-play. Semantics first, then syntax: new syntax is earned by demonstrated semantics, never invented ahead of it. The trust story in one sentence: **the generator never writes a result; only the kernel does, by running judges over transported evidence.**

The kernel ships with zero languages, zero transports, zero domains. Presented a benchmark, the first admissions are the domain — root name plus anchors, the ungenerable half — then the root language's interpreter, then a first naive search in pure Python. Witnesses need no declared form, so existential questions can settle *certified* with a single admitted interpreter; the first certificate form and checker arrive when universal questions need better than *claimed*. [Section 5](#) reports that this is how the tree was in fact populated.

4 THE KERNEL, MADE SOLID FOUR WAYS

The kernel is the fixed part: the gate, the sealed runner, the result order, the registry loader, and the driver that plays, regrades, and renders — five stdlib-only Python modules of 2,654 lines at the tag (driver.py 689, gate.py 1,238, registry.py 191, results.py 419, runner.py 85, and a 32-line package file). It is the one piece of code the gate cannot judge, because it is the gate; it is generated like everything else in the tree; and so four disciplines stand in for the gate, each worded no stronger than what it verifies.

It computes no truth of its own. The kernel runs judges, compares bytes, orders results, and draws the log. A kernel bug can lose a result or misgrade one; the property that it can never forge one — no record reaches *certified* unless an interpreter run where the question lives passed, whatever a search, a carry-back, or a translator wrote — is stated in the specification and falsified by test (kernel/tests/test_forge.py). On a toy registry bootstrapped from empty through the real gate, a search that prints a witness together with the words "grade": "certified" never becomes a result, because its witness does not replay; a bogus certificate floors at *claimed*; a broken carry-back only loses the witness; a route missing a carry-back is a partial, never an answer; and regrade lifts a stored proof to *certified* by check time alone once the pair learns to carry certificates. [Section 6.1](#) asks the same question of the real registry.

The half that is mathematics is proved. The Lean development under kernel/mechanization/[13] — 502 lines, no dependencies beyond the Lean core, toolchain pinned — proves, for exactly the key of [Section 3.5](#), the properties the specification asks for. A key is $(level, bound, grade, gap)$ with $gap \in \mathbb{N} \cup \{\perp\}$; write $k < k'$ for the lexicographic order in which a smaller finite gap is better and $\perp$ (no judgment ever ran) is worst, and $best(\ell)$ for the fold of a log ℓ that keeps the incumbent only while it is strictly better.

THEOREM 4.1 (THE ORDER AND THE RATCHET). *$<$ is irreflexive, transitive, and asymmetric. For all logs ℓ, ℓ' : $best(\ell) \leq best(\ell \# \ell')$; if $best(\ell)$ is settled then so is $best(\ell \# \ell')$; and if the two have the same level and bound, then the grade of the second is no lower and, at equal grade, its gap no larger. A record that differs from another only by a higher grade, or only by a smaller gap, is strictly above it.*

THEOREM 4.2 (THE RESIDUAL). *Let $h_1, \dots, h_n$ be the lineages of a route's hops, j the lineage of the judge that ran, and $residual(h, g, j) = \bigcup_{i \leq g} h_i \cup j$. Then $residual(h, 0, j) = j$; $g \leq g'$ implies $residual(h, g, j) \subseteq residual(h, g', j)$; for $g \leq n$, appending hops upstream of the judgment leaves the residual unchanged; and every element of a residual belongs to the judge or to a hop, never to the search alone.*

The first clause of [Theorem 4.2](#) is what makes *certified* route-independent: at gap 0 the result rests on the home judge and on nothing that carried the question out or the evidence back. The last clause of [Theorem 4.1](#) is why a regrade, which keeps the value, is a strict improvement exactly when it moves the grade or the gap. The build prints the axiom audit for every theorem: the order lemmas are axiom-free; the fold and residual lemmas rest on `propext` and `Quot.sound` through the core library's list lemmas. What the model does not cover is said in its README so the layout never overstates: the proofs are about the order and the residual as functions of keys and lineages, not about the Python that computes them — that correspondence is the job of `test_order.py`, which runs the same properties against `results.key` on random records.

The half that is operation is falsified. The test suite (`python3 -m kernel.tests`; 2,259 lines in nine modules and the toy registry) is the gate turned on the kernel. `test_gate` runs the gate against the registry's own controls: every bound entry's stamp is re-derived by re-running its admission — its agreement counts a floor, since a stamp is never rewritten and a pair may join after the language it binds to was stamped, an allowance pinned the first time it was needed — and every supplied mutant is refused again; and on the toy registry every way of failing the gate fails it — a language without controls, a mutant that passes, a judge that accepts anything, an implementation that reaches for a tool, nondeterminism, a revision that disagrees with its predecessor, a search that only abstains. `test_seal` measures the seal: the environment holds nothing but a blank PATH, the working directory is scratch and gone afterwards, no tool can be found — not even on the platform default path — determinism is measured rather than declared, and a hit wall is a result, not an exception. A test that found the seal weaker than the specification claimed — an unset PATH still found `/usr/bin/python3` — is how the blank PATH got into the specification. `test_registry` recomputes the content pin from its definition for every stamp and refuses changed bytes, pinless stamps, and two entries claiming one name at one revision. `test_misaligned` exercises the frame contract of [Section 3.2](#) end to end, mutants included. `test_regenerate` regenerates the board and the graph of every pinned run byte-identically from the log, and `test_faults` holds the fault-injection harness of [Section 6.1](#) to what it must be able to tell apart.

A second lineage agrees. `kernel/second/` is the kernel's pure half — registry loading with pin verification, benchmark pins, the order, best-per-question, the frontier, contradictions, corroboration, the board, the graph, the base; 727 lines — generated clean-room from the specification and the committed data alone, never from the first kernel's source. Its README records the protocol: what was read (the specification, the manifests, the runs, directory listings), what was run (the first kernel only as a black box through its command line; never `play`, `admit`, or `regrade`), one disclosure (an early traceback on a corrupt registry named two source files and line numbers; the first kernel now refuses a corrupt registry without a traceback), and where the specification left it guessing — eleven listed ambiguities on inputs no pinned run exercises. It is held to byte-agreement with the first kernel on base, report, and graph over every pinned run and over a synthetic run holding the corners no pinned run exercises (`test_second`), and a runtime audit hook on open confirms that no module of it reads any file of the first kernel (the import system loads the bytecode of the kernel package's own `__init__`, because `second` is its subpackage, and nothing else). This is the accelerator discipline of [Section 3.6](#) turned on the kernel: two implementations of disjoint descent that agree on every byte corroborate each other, as two judges do. Where they disagreed while being built — the key's tie rule, the pin's exclusions, the binding rule for revisions — the specification was made to say which is right, and now does.

Manual review is the last discipline, and the four above are what make it tractable: what must be read is exactly the trusted base — the admitted judges, printed by base ([Section 5](#)), and the kernel's

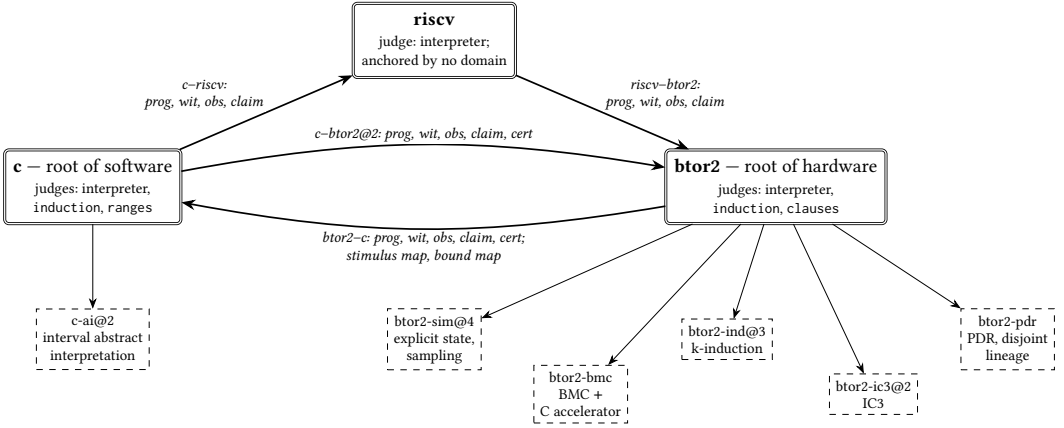


Fig. 3. The registry at the tag: three languages (double borders), four pairs (solid arrows, labelled with the artifacts they carry), six searches (dashed, at the language they live at). Names carry their highest admitted revision. Every node interprets; every edge translates; every search writes. Routes never revisit a language, so the two pairs between c and btor2 never play the round trip.

Table 1. The trusted base at the tag, as `python3 -m kernel.driver base` prints it — judges only; everything else in the registry is untrusted syntax. Vectors are programs with pinned observables; controls are mutants the gate refused. For a checker, vectors are certificates that must discharge and controls certificates that must not. Sizes are lines of Python.

judge	kind	vectors	controls	lines	corroborated by
btor2@6	interpreter	18	10	390	hardware (3 anchors)
btor2:induction	checker	18	12	1,620	
btor2:clauses	checker	10	10	1,618	
c@7	interpreter	17	10	1,422	software (3 anchors)
c:induction	checker	46	25	4,534	
c:ranges	checker	42	21	3,935	
riscv@2	interpreter	20	12	603	no domain; its pairs' squares only

five modules — and the reviewer reads theorem statements, tests, and a second implementation's disagreements rather than a story.

5 WHAT WAS ADMITTED

Everything under `registry/` at the tag entered through the gate of Section 3.7, in the order the conjecture order prescribes, and every entry carries its stamp. Figure 3 draws the registry; Table 1 is the trusted base as the kernel prints it.

The triangle. The hardware domain entered first, before its root interpreter existed: root btor2, with the HWMCC'24 bit-vector track and the official verdicts of its solvers as recorded testimony. Then the BTOR2 interpreter, the trusted floor; the naive first citizen, btor2-sim, exhaustive enumeration with fixpoint detection when the free-input space is small and guided sampling otherwise; and btor2-bmc, bit-blasting per frame to an and-inverter graph decided by a generated CDCL SAT solver, every model replayed before it is emitted, with an accelerator in C admitted by byte-agreement on its corpus. The software domain and the C interpreter followed — the

preprocessor-free ILP32 integer fragment SV-COMP’s loop and bit-vector tasks are written in, its semantics total and deterministic — then RISC-V as the middle vertex, anchored by no domain, and the three pairs `c-btor2`, `c-riscv`, `riscv-btor2`, each carrying programs, witnesses, observables, and claims, with mutants per artifact kind and exchange rates of 3.65, 5.37, and 3.13 target bytes per source byte. Two certificate forms then arrived on BTOR2 by revision of the language — induction (bit-invariants and k -induction) and clauses (a conjunction of clauses over state bits, as property-directed searches converge to) — with the searches that write them, `btor2-ind` and `btor2-ic3`; then C’s own induction checker and the certificate carry-back of the C-to-BTOR2 pair, by revision; and a further revision of C’s checker that builds its machine by the RISC-V road, sharing no text with the C-to-BTOR2 encoding, so that a proof which travelled that pair is certified by an encoding the pair never touched.

The reverse edge. C gained a third certificate form, ranges — per program point a box over the scalar slots, unlisted points asserted unreachable — and its first search, `c-ai`, interval abstract interpretation that writes such a certificate or an honest partial and never a bare claim. On the frame contract of [Section 3.2](#) the pair `btor2-c` was admitted with all five channels: every state a global of the smallest unsigned container holding its width, one statement per node, `ite` branch-free, so that a bad at machine frame t is observed at C frame $n_0 + n \cdot t + bad$. Seventeen corpus machines, six certificates carried both ways, thirteen mutants refused, 4.19 bytes per byte; array sorts, widths above 64, and non-constant initial values are refused loudly.

Three more searches. `btor2-sim@4` carries arrays as sparse maps whose cells are drawn as they are read. `btor2-pdr` is a second property-directed search [\[15\]](#) written from nothing it shares with `btor2-ic3` — its own parser, bit-blaster, and SAT solver — so that its lineage is disjoint from every earlier search’s; seven corpus machines, three certificates discharged, three mutants refused. `c-ai@2` reads the reverse edge’s branch-free `ite` as the selection it is and adds a case split.

Revisions that add only vectors. An admitted entry is never edited, and a revision need not change a line of code. After the measurement of [Section 6.1](#) each language was revised with vectors and controls alone — `btor2@6`, `c@6`, `c@7`, `riscv@2` — [Section 6.2](#) says with what. Fifteen language entries, five pair entries, thirteen search entries, and two domains stand in the tree; a name binds to its highest revision, and the earlier revisions tell the system’s small history without editing anything: `btor2-ind@2` and `btor2-ic3@2` blasted the signed division family after thirteen SV-COMP programs and four HWMCC machines crashed the first revisions, each byte-identical to its predecessor on the predecessor’s corpus.

The generator. Every implementation in the registry, and the kernel, was written by language-model agents working at the command line the loop describes ([Section 1](#), Provenance); the entries declare four generator tags among their lineages. A human chose the benchmarks, set the goal of each campaign, and pulled the plug. Two things the tree does not record limit what can be said about the generator itself: an entry that fails the gate leaves no stamp and no trace, so the rate at which generated candidates were refused during development is not known; and nothing was generated twice from the same instruction, so the variance of the generator is not measured. [Section 6.1](#) measures what can be measured after the fact: how the gate and the grades respond to faults placed deliberately.

6 MEASUREMENTS

Every number in this section is read from the tree at one tag. Three things are measured: what faults placed deliberately do to the gate and to the grades ([Section 6.1](#)); what it took to close what they found ([Section 6.2](#)); and what the system settles on four pinned benchmarks ([Section 6.3](#)).

6.1 Fault injection

Every entry ships its own mutants and the gate refuses them; that says the controls can fail, not how much they see. We therefore mutate the registry ourselves, in the manner of mutation testing [14, 19]. A *mutant* is one executable with one classical fault: a comparison or an arithmetic or bitwise operator replaced, and and or exchanged, a negation dropped, a branch condition negated, an integer constant moved by one or a truth constant flipped, or a statement with no binding effect removed. Sites are enumerated over the syntax tree of every executable a name binds to — the seven judges and the eighteen transports (twelve translators, carry-backs, and maps of the four pairs; six searches) — and one hundred per file are drawn by a hash of the site, so the sample is reproducible; two small files have fewer sites, giving 700 judge mutants and 1,672 transport mutants. A mutant is staged in a scratch copy of its entry and run sealed like everything else, under a wall of four times the intact entry’s longest run and a memory cap; the registry and the logs are never written. Each experiment first runs the intact file and its unparsed-and-reparsed identity, which must behave as the entry does.

At the gate. A mutated entry faces what an entry of its kind faces at admission, in order, and the first refusal is recorded: its own vectors, corpus, and controls; *conservativity*, byte-agreement with its predecessor on the predecessor’s checkable surface; and, for a judge only, its *dependents* — the admission of every pair and search that runs the judge, re-run against the mutant, which is what Section 3.1 means by a judge being corroborated after it entered. Of the 1,672 transport mutants the gate refuses 736 (694 by the entry’s own gate, 42 by conservativity) and passes 936, of which 528 sit on lines no admission input executes. That is the design’s free surface: what a transport writes still faces judges at play time, and the next experiment measures what the 936 can do. Of the 700 judge mutants the gate refuses 348 — 292 by the judge’s own vectors and controls, 26 by its predecessor, 30 by dependents (Table 2, left). Two things in that count bear on the design. A revision is a differential oracle: 25 faults in BTOR2’s interpreter pass its vectors and are caught only by disagreement with the previous revision. And the claim that RISC-V, anchored by no domain, is held in place by the pairs on either side of it has a number: eleven faults in its interpreter pass its own vectors and are refused only by those pairs’ squares. The 352 survivors are the trusted base’s exposure, and they sit where the judges are large: the interpreters refuse 61 to 80 of 100, the checkers 26 to 49, and admission executes between 44% and 70% of a checker’s statements — most of the rest a bit-blaster that no shipped certificate’s machine exercises.

With the gate taken away. The design’s central claim is not about the gate. It is that a transport, being judged, can lose a grade and never forge one, and that the residual names what a result rests on (Section 3.4). To test it we remove the gate: each of the 1,672 transport mutants is put where the intact file stood, as if admitted, and the kernel plays the routes through it on questions whose answer is known — the labelled corpus questions of the searches, and questions of the pinned runs for which the board holds a replayed witness or a universal answer certified at gap 0. For each transport ten (question, route) pairs that the intact entry plays within two seconds are chosen by hash, in turn from four kinds so that a wrong answer has somewhere to show: unsafe and answered by a witness, safe and answered by a judged proof, unsafe and unanswered, safe and unanswered — 180 pairs, half of them unsafe. Each record the kernel writes for a mutant is then held against the truth and against the intact route’s record (Table 3).

No play leaves a wrong record *certified*. The 127 wrong *claimed* records are what the lowest grade is for: a search whose comparison is inverted answers “safe forever” on an unsafe machine, nothing judges the claim, and its residual is the whole chain, the search included. The six wrong *checked* records are the instructive ones. All come from mutants of the C-to-BTOR2 translator on

Table 2. The judges under fault injection, 100 sampled mutants each. *Before*: at the revisions the boards of Section 6.3 were first played under. *After*: the identical mutants against the vector-only revisions of Section 6.2. Statements: the share of the judge’s statements executed by any admission input. Laxer: gate survivors that discharge a certificate the intact checker refuses, on the pool of the probe.

judge	before			after		
	statements	refused	laxer	statements	refused	laxer
btor2 interpreter	90%	80	—	96%	98	—
btor2:induction	54%	33	4	61%	43	0
btor2:clauses	44%	26	1	52%	42	0
c interpreter	79%	61	—	89%	83	—
c:induction	63%	37	1	69%	43	0
c:ranges	70%	49	0	75%	55	0
riscv interpreter	76%	62	—	93%	89	—
all seven, of 700		348	6		453	0

Table 3. 16,720 plays of 1,672 transport mutants forced past the gate, each record held against the known answer. *Wrong*: the record settles the question the other way — a universal claim covering the ask where a witness is known, or a witness within the bound a question is known safe to.

mutants of	same	lost	gained	wrong, by the grade it carries		
				<i>claimed</i>	<i>checked</i>	<i>certified</i>
the pairs’ transports (1,072)	9,387	1,312	14	1	6	0
the searches (600)	4,769	1,056	49	126	0	0
all (1,672)	14,156	2,368	63	127	6	0

one unsafe SV-COMP task: the mistranslated machine really is safe, the property-directed search proves it, BTOR2’s checker discharges the certificate — and the certificate, carried home, fails to discharge at C, so the gap stays at one and the residual names the translator. In all 133 wrong records the residual contains the faulty entry’s whole lineage. Read against the first experiment: of the 736 mutants the gate refuses, 46 would have answered wrongly on some question and 506 would have lost an answer; of the 936 it passes, 832 change nothing on any played question, 90 lose an answer, and 7 answer wrongly — one of them a translator mutant behind a wrong *checked* proof. The gate missed those seven and the grade still said what each answer rested on.

Which way a surviving checker leans. For a judge there is no judge downstream, so a surviving mutant has to be classified another way. A checker mutant the gate passes is the same judge, a stricter one — it refuses something the intact checker discharges, which costs answers and no trust — or a *laxer* one, which discharges something the intact checker refuses; only the last is a hole. We hold each of the 255 surviving checker mutants against the intact checker on a pool of up to sixty (program, certificate) pairs per checker, drawn from everything in the tree that holds a certificate of its form — its vectors and controls, what the admitted searches write on their corpora, what the pairs carry, the certificates in the logs of the pinned runs — each certificate also set against programs it was not written for and edited once (an integer moved by one, a list element or a dictionary entry dropped). On that pool 245 agree with the intact checker everywhere, 4 are stricter, and 6 are laxer: in BTOR2’s checkers, neq bit-blasted as eq, a signed comparison reading one operand twice, and

three faults in the embedded SAT solver’s propagation and assumption handling; in C’s induction checker, a zero-extending shift dropped from the machine it builds. “Same” is a statement about the pool, not a proof of equivalence: most of the 245 sit in code that no certificate we could find reaches.

6.2 Closing the survivors

A survivor on a line that admission executes names a missing test, and the design has one way to add a test: a revision (Section 3.7). The four revisions made after the measurement change no line of any judge. What they add is vectors and controls, and since a vector’s expected value written by reading the judge would test nothing, every expected value was computed from the relevant standard by a small generator and then put to an *oracle* in the sense of Section 3.6, a tool that runs outside the seal and never enters the registry:

- BTOR2: 301 checks, each one operator on constants against the value SMT-LIB’s definitions give, the conjunction of a family as the bad property. BtorMC [33] finds every check reachable as expected and unreachable negated. The same tables over states frozen at the constants, bad negated, are certificate vectors for both checkers: the certificate is the invariant that pins every state bit, its control that invariant made wrong.
- RISC-V: 896 checks, each one instruction or short idiom against the value the specification gives, assembled bare-metal and run on the executable of the Sail model [3]. Every table succeeds as written, and with any one expected value moved by one fails with that check’s number.
- C: assertion programs for the interpreter, and for the checkers 37 programs that never fail — operator tables at the corners of each integer type, division in programs of two variables, identities over all pairs of bytes — with 651 checks that clang confirms as written and refutes with any one expected value moved. A certificate for a C program is discharged on a machine the checker builds and unrolls, so the programs are small; each ranges certificate is what c-ai wrote, each induction certificate k -induction at the smallest k that discharges.

No oracle disputed a judge: on every new check the generated interpreters and checkers were right, and the corners had simply never been observed. (The oracles did catch five wrong expectations in our own generator, which is what they are for.) Where the fragment stipulates a meaning the standard leaves undefined — a shift count at or beyond the width, a zero divisor — the vector pins the stipulation and no oracle is asked. Table 2, right, repeats the first experiment on the identical mutants. The interpreters now refuse 98, 83, and 89 of 100, and every one of the thirty that remain is an equivalent mutant, a removed refusal — which a vector, a program the interpreter must run, cannot observe — or one ordering of nested array values no vector builds. The checkers refuse 42 to 55 of 100; all six laxer mutants are refused by the new vectors, and none of the 217 mutants that still survive is laxer on the pool. What is left there sits mostly in code a certificate check never reaches: each checker embeds whole an interpreter, an array engine, or translator cases for instructions its own road never emits. That is not exposure so much as weight a judge should not carry, and it is the next thing the measurement asks for. The vectors are not free: admitting C’s language entry now takes a little over two minutes where it took under one.

6.3 The boards

Protocol. Four benchmarks are pinned, every program by sha256 (Table 4). Three are tiers of the HWMCC’24 bit-vector track [18, 33]: hwmcc24-mini (74 machines, 23% of the track), hwmcc24-arrays (55 machines with array sorts), and hwmcc24-mid (80). The fourth, svcomp25-mini, holds 79 ReachSafety loop and bit-vector tasks from SV-COMP 2025 [9], 31 of

Table 4. The pinned benchmarks and the boards at the tag. Settled: a replayed witness, or a universal claim covering the ask. Certified universal answers are at gap 0; every witness is certified by replay at the root; covering claims are bare, graded *claimed*. Corroborated: settled verdicts on which two results of disjoint lineage agree. Official verdicts are read from the competition archives; unsolved means no official solver answered.

benchmark	asked	settled	of which			corr.	open	open, officially	
			cert. $\forall$	witness	claim			unsolved	solved
hwmcc24-mini	74	45	24	9	12	29	29	7	22 (16 unsat, 6 sat)
svcomp25-mini	79	26	6	14	6	0	53	0	53 (30 true, 23 false)
hwmcc24-mid	80	47	19	8	20	22	33	8	25 (17 unsat, 8 sat)
hwmcc24-arrays	55	12	0	12	0	0	43	38	5 (3 unsat, 2 sat)

them tasks that at most three of the verifiers in the archive answered correctly. An ask exists bad at a bound, or forall bad at a bound or at ∞ . A play runs every question along every admitted route whose fragment admits it, one wall per route — 60 s, except two early software plays at 30 s and one pass at 600 s named below — and a search that runs to twice its wall is killed and booked as a partial saying so. After an admission only the routes the new entry opens are played, and the log records which.

The first plays. With the triangle of Section 5 and four BTOR2 searches, hwmcc24-mini closed at 43 of 74 and svcomp25-mini at 26 of 79. Twelve of the fourteen software witnesses walked home across both bridges of the triangle, found at BTOR2 on the route C to RISC-V to BTOR2 and replayed by I_C . Three universal answers on software were first *checked* at gap 1 — k-induction certificates discharged by BTOR2’s checker, their residual naming the C-to-BTOR2 pair — and then, once C shipped its own induction checker and the pair gained a certificate carry-back by revision, lifted to *certified* at gap 0 by regrade alone, without re-solving anything, in 0.55, 0.41, and 0.51 s.

Three negatives. The reverse edge of Section 5 opens one new route on every hardware question, BTOR2 to C to c-ai. Played alone, all 74 routes on hwmcc24-mini came back partials, and the log says why: on 41 machines intervals see the error reachable, because the pinned branch-free ite encoding — $b \oplus ((a \oplus b) \wedge -c)$ — is opaque to a box; 17 translations are refused at widths above 64; 12 questions are existential, which a proving search never refutes; 4 budgets end before a fixpoint. The two larger rungs, played for the first time, settled 47 of 80 and 1 of 55: on the array rung the explicit-state search, IC3, and the reverse edge all refused array sorts, and only bounded model checking and k-induction ran at all. And IC3 alone with a ten-minute wall on the ten open universal questions of hwmcc24-mini proved one — a 64-bit machine, certified at gap 0 in thirty seconds once its budget let it go deep enough — and was killed at twice the wall on six. Ten times the clock bought one question: the boards were search-limited, not budget-limited.

One turn of the loop. The conjecture those negatives support is the (a)-move of Section 3.8, new searching at existing languages, and three searches were generated, gated, and played through their own routes. btor2-sim@4, which carries arrays as sparse maps, moved the array rung from 1 to 12 of 55: eleven witnesses, each certified by replay. btor2-pdr certified 22 universal answers on hwmcc24-mini and 17 on hwmcc24-mid; one question was newly settled, and fifteen that the boards had held only as bare claims became certificates at gap 0, as did one on software, where the search reaches C questions over the C-to-BTOR2 pair and its certificates discharge at C. Over the RISC-V road every one of its 79 software routes is a partial, because that road models memory as an

array and the search refuses array sorts; on eleven machines of the middle rung its transition cone exceeds its own cap. Because its lineage is disjoint from every earlier search's, the *corroborated* flag fires for the first time: 29 verdicts on hwmcc24-mini and 22 on hwmcc24-mid are held by two searches that share no descent — two searches, not two judges; every such answer still rests on BTOR2's one lineage of judges, and no software verdict is corroborated, since every route to a C question shares the C front end. c-ai@2, which reads the reverse edge's *ite* as the selection it is, moved nothing: all 154 of its routes on the two rungs are partials. One conjecture of three was wrong, one moved a count, one moved grades; the log records all three the same way.

Against the official verdicts. The competitions' verdicts are anchors: recorded per question with provenance, drawn beside the board, never ranked, never a judge. All 45, 26, 47, and 12 settled questions agree with them, with one row that is not the same answer: an existential question on the middle rung settled in the negative by a bare claim to bound 30 that five official solvers record as satisfiable at a depth the archive does not publish — a bounded claim beside an unbounded verdict, not a contradiction. Of the open questions, 53 on the three hardware boards were solved by no official solver, and 105 were: bounded safety questions on which the bounded searches stop short of the ask, existential questions whose witnesses the searches did not find, and every one of the 53 open software tasks, including all 31 selected because at most three verifiers answered them. No board holds a replayed witness or a discharged certificate whose official verdict disagrees with it, and none holds a question settled here that the archive records as unsolved.

6.4 What the numbers say

Four readings are supported, and no more. First, the no-forge claim survives a test designed to break it: with the gate removed, 1,672 faulty transports produce wrong answers at the two grades that say “not judged at home” and at no other, and every wrong answer's residual names its cause. Second, the trusted base is measurably the weak part and measurably improvable: half of the sampled judge faults passed admission, six were real holes, and vector-only revisions with outside ground truth closed all six and raised the refusal count from 348 to 453 of 700 — while showing that what remains is mostly code the checkers should not contain. Third, the geometry does what it was built to do on real questions: 49 universal answers across the boards are *certified* at gap 0 with the home judge alone in their residual, sixteen of them lifted from bare claims by a search admitted later, three by a regrade that re-solved nothing; the 38 covering claims that remain are *claimed*, because nothing judged them. Fourth, the system is far from the solvers it records beside its boards: it settles no question they left unsolved and leaves 105 open that they closed, and its negatives — an edge blind to an idiom, a road refused for its arrays, a wall that bought one question — are structural and are in the log.

7 RELATED WORK

Generated verification infrastructure. Language models are used to write proofs and repair them until a checker accepts them [16, 45, 46], to propose invariants for a verifier to confirm [34, 44], to synthesize code together with its specification and proof [1, 29], and to keep code, documentation, and tests consistent under a checker's eye [39]. In every case the checker is fixed and trusted and the model fills one slot. The system here lets the model fill every slot — interpreter, translator, checker, search, carry-back — and moves trust to a property no slot can assert: where along a route an interpreter run judged the evidence. Kirsch [21] gives a calculus of directional squares, a componentwise meet for composed translations, and the asymmetry between replayed witnesses and universal claims, for a platform in which engines are wrapped and registration is done by hand; the design here consumes that calculus and shares nothing else with that platform.

Translation validation and verified compilation. Checking a translation per program rather than proving the translator [32, 35] is the square of Section 3.2, with two differences: both sides are generated interpreters rather than a trusted semantics, and the check is on a shipped corpus at admission with mutants that must fail, not on every compilation. Verified compilers [24, 25] and verified translation validators [37, 42] buy a stronger statement at the cost of a proof per translator; Alive2 [26] bounds the check the way a bound cap bounds a claim here. The system does not prove any translator and says so; what it adds is that the grade of a result never depends on whether a translator was right, only on whether a judge ran.

Certificates, witnesses, and proof formats. Certifying model checkers [30], certified hardware model checking [48], verification witnesses [7, 8], proof-carrying code [31], certifying algorithms [27], and checked proof formats [2, 40, 43] share the move that an untrusted producer emits an object a small checker validates. Here the checker is generated too, admitted by its own controls, and the object’s grade records how many translations lie between the checker that validated it and the question; the absence of a certificate is also data, a *claimed* result being a standing demand for the printer that would upgrade it. Certified abstract interpretation [5, 20] produces or verifies the kind of certificate `c`: ranges checks; the difference is that the analysis here is untrusted and the checker is what was gated.

The searches. The generated searches implement known algorithms: bounded model checking [10], k-induction [38], IC3 and property-directed reachability [11, 15], interval abstract interpretation [12]; none is claimed to be competitive with the solvers whose verdicts sit beside the boards. Btor2C [6] translates BTOR2 machines to C so that software verifiers reach hardware questions; the reverse edge of Section 5 is the same move, with the translation gated by squares in both directions and its bound map judged against both interpreters — and its result at the tag is a negative.

Mutation analysis. Mutation testing measures a test suite by the faults it detects [14, 19]. Section 6.1 turns it on a verification system in three ways that the design makes available: the “test suite” a mutant faces is the admission gate, with a predecessor revision and downstream admissions as further oracles; a mutant of an untrusted component can be executed past the gate and its outputs scored by the grade the system itself assigns; and a surviving mutant of a checker is classified by direction, since only one direction is unsound. Every entry also ships hand-chosen mutants as controls, which is the same idea at admission time.

The trusted base. That trust cannot be bootstrapped from a component one did not judge is old [41]; the LCF discipline of a small kernel every proof must pass [17] is the shape of the trusted base here, with the kernel’s own standing established by proof, tests, a second lineage, and pins rather than by being small enough to believe. Differential testing [28, 47] and N-version programming [4] supply the second-lineage discipline, and the finding that independently written versions fail together [23] is why the corroborated flag requires declared disjoint descent and is, at the tag, earned only between two searches. An evidential tool bus [36] records the evidence tools produce; here the tools are not trusted to produce it, and what is recorded is where an interpreter judged it. Rotor [22] fixes one translation from RISC-V to BTOR2 by hand; the pairs `c-riscv` and `riscv-btor2` are translations of that kind generated and gated, with the ISA semantics generated rather than taken from a specification language [3] — whose model serves here as an oracle for the generated interpreter’s vectors instead.

8 LIMITATIONS AND CONCLUSION

The limits are the ones the tree states. The square is empirical: a pair is admitted when its squares close on its corpus, and a corpus program is not every program; the mutants make the corpus

falsifiable, not complete. The seal is a blank PATH and an emptied environment, measured by tests; an absolute path to a tool is hidden by nothing but the committed source that would show it. The judges are not small — C’s two checkers are 4,534 and 3,935 lines, because each embeds the front end, the RISC-V-road encoding, and the bit-blaster it judges with — and the measurement of Section 6.1 says what that costs: after every revision, 247 of 700 sampled judge mutants still pass admission, and for the 217 of them that are checker mutants the evidence that they are harmless is a pool of certificates, not a proof. The fault model is first-order syntactic mutation, a hundred sites per file, ten questions per transport; a fault that needs two edits, or a question we did not play, is outside it. The corroborated flag has fired between two searches of disjoint descent and never between two judges: every answer still rests on one lineage of interpreter and checker per language. Nothing was generated twice from one instruction, and candidates the gate refused during development left no trace, so neither the generator’s variance nor its failure rate is measured. The reach table, the oracle column, and the per-domain ledger are specified and not drawn. And the generated searches are weak beside the competitions’ solvers: no board holds a question those solvers left unsolved.

Within those limits the claim is exact in kind. A verification system can be built entirely from generated, untrusted text — interpreters, translators, searches, checkers, carry-backs, and the kernel itself — and judged whole: one gate admits everything by running it against its own controls; one trust event, an interpreter run, is the only source of truth; a result’s grade is the distance from the question to the last such run, and its residual is computed by removing everything upstream of that run. The claim that follows from the design — that a faulty transport can cost an answer and cannot forge one — held on every one of 16,720 plays of deliberately faulty transports with the gate removed. The claim that does not follow from the design, that the judges deserve their place, was measured instead of asserted: it failed in six places, each was found by the system’s own instruments, and each was closed by adding tests whose answers came from outside. At one frozen commit the system holds four boards whose every judged answer agrees with the competitions’ verdicts and 49 universal answers that rest on a home judge alone. The generator never writes a result; only the kernel does, by running judges over transported evidence. What the measurement asks for next is smaller judges, and a second lineage of them.

REFERENCES

- [1] Pranjal Aggarwal, Bryan Parno, and Sean Welleck. 2025. AlphaVerus: Bootstrapping Formally Verified Code Generation through Self-Improving Translation and Treefinement. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (PMLR, Vol. 267)*. 587–615.
- [2] Bruno Andreotti, Hanna Lachnitt, and Haniel Barbosa. 2023. Carcara: An Efficient Proof Checker and Elaborator for SMT Proofs in the Alethe Format. In *TACAS*.
- [3] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E. Gray, Robert M. Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, Shaked Flur, Ian Stark, Neel Krishnaswami, and Peter Sewell. 2019. ISA Semantics for ARMv8-A, RISC-V, and CHERI-MIPS. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 71:1–71:31.
- [4] Algirdas Avizienis. 1985. The N-Version Approach to Fault-Tolerant Software. *IEEE Transactions on Software Engineering* SE-11, 12 (1985), 1491–1501.
- [5] Frédéric Besson, Thomas Jensen, and David Pichardie. 2006. Proof-Carrying Code from Certified Abstract Interpretation and Fixpoint Compression. *Theoretical Computer Science* 364, 3 (2006), 273–291.
- [6] Dirk Beyer, Po-Chun Chien, and Nian-Ze Lee. 2023. Bridging Hardware and Software Analysis with Btor2C: A Word-Level-Transition-System-to-C Translator. In *TACAS (LNCS)*.
- [7] Dirk Beyer, Matthias Dangl, Daniel Dietsch, Matthias Heizmann, Thomas Lemberger, and Michael Tautschnig. 2022. Verification Witnesses. *ACM Transactions on Software Engineering and Methodology* 31, 4 (2022).
- [8] Dirk Beyer, Matthias Dangl, Daniel Dietsch, Matthias Heizmann, and Andreas Stahlbauer. 2015. Witness Validation and Stepwise Testification across Software Verifiers. In *ESEC/FSE*.
- [9] Dirk Beyer and Jan Strejček. 2025. Improvements in Software Verification and Witness Validation: SV-COMP 2025. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS), Part III (LNCS, Vol. 15698)*. 151–186.

- [10] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. 1999. Symbolic Model Checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS, Vol. 1579)*. 193–207.
- [11] Aaron R. Bradley. 2011. SAT-Based Model Checking without Unrolling. In *VMCAI (LNCS, Vol. 6538)*. 70–87.
- [12] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *POPL*. 238–252.
- [13] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *CADE (LNCS, Vol. 12699)*. 625–635.
- [14] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. 1978. Hints on Test Data Selection: Help for the Practicing Programmer. *Computer* 11, 4 (1978), 34–41.
- [15] Niklas Eén, Alan Mishchenko, and Robert Brayton. 2011. Efficient Implementation of Property Directed Reachability. In *FMCAD*. 125–134.
- [16] Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. 2023. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *ESEC/FSE*.
- [17] Michael J. C. Gordon, Robin Milner, and Christopher P. Wadsworth. 1979. *Edinburgh LCF: A Mechanised Logic of Computation*. LNCS, Vol. 78. Springer.
- [18] HWMCC. 2024. Hardware Model Checking Competition 2024. <https://hwmcc.github.io/2024/>. Benchmarks and results archive.
- [19] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering* 37, 5 (2011), 649–678.
- [20] Jacques-Henri Jourdan, Vincent Laporte, Sandrine Blazy, Xavier Leroy, and David Pichardie. 2015. A Formally-Verified C Static Analyzer. In *POPL*. 247–259.
- [21] Christoph Kirsch. 2026. Untrusted Authors, Trusted Answers: A Calculus of Fidelity-Graded Translations. arXiv preprint arXiv:2607.14137, v2.
- [22] Christoph M. Kirsch. 2017. Selfie and the Basics. In *Onward!*
- [23] John C. Knight and Nancy G. Leveson. 1986. An Experimental Evaluation of the Assumption of Independence in Multiversion Programming. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 96–109.
- [24] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. 2014. CakeML: A Verified Implementation of ML. In *POPL*.
- [25] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (2009), 107–115.
- [26] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. In *PLDI*.
- [27] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161.
- [28] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.
- [29] Md Rakib Hossain Misu, Cristina V. Lopes, Iris Ma, and James Noble. 2024. Towards AI-Assisted Synthesis of Verified Dafny Methods. *Proceedings of the ACM on Software Engineering* 1, FSE (2024).
- [30] Kedar S. Namjoshi. 2001. Certifying Model Checkers. In *CAV*.
- [31] George C. Necula. 1997. Proof-Carrying Code. In *POPL*.
- [32] George C. Necula. 2000. Translation Validation for an Optimizing Compiler. In *PLDI*.
- [33] Aina Niemetz, Mathias Preiner, Clifford Wolf, and Armin Biere. 2018. Btor2, BtorMC and Boolector 3.0. In *CAV*.
- [34] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2023. Can Large Language Models Reason about Program Invariants?. In *ICML*.
- [35] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *TACAS*.
- [36] John Rushby. 2005. An Evidential Tool Bus. In *ICFEM*.
- [37] Thomas Arthur Leck Sewell, Magnus O. Myreen, and Gerwin Klein. 2013. Translation Validation for a Verified OS Kernel. In *PLDI*.
- [38] Mary Sheeran, Satnam Singh, and Gunnar Stålmarck. 2000. Checking Safety Properties Using Induction and a SAT-Solver. In *FMCAD (LNCS, Vol. 1954)*. 108–125.
- [39] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. 2024. Clover: Closed-Loop Verifiable Code Generation. *arXiv preprint arXiv:2310.17807* (2024).
- [40] Yong Kiam Tan, Marijn J. H. Heule, and Magnus O. Myreen. 2021. cake_lpr: Verified Propagation Redundancy Checking in CakeML. In *TACAS*.
- [41] Ken Thompson. 1984. Reflections on Trusting Trust. *Commun. ACM* 27, 8 (1984), 761–763.
- [42] Jean-Baptiste Tristan and Xavier Leroy. 2008. Formal Verification of Translation Validators: A Case Study on Instruction Scheduling Optimizations. In *POPL*.
- [43] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt Jr. 2014. DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs. In *SAT*.

- [44] Haoze Wu, Clark Barrett, and Nina Narodytska. 2024. Lemur: Integrating Large Language Models in Automated Program Verification. In *ICLR*.
- [45] Chenyuan Yang, Xuheng Li, Md Rakib Hossain Misu, Jianan Yao, Weidong Cui, Yeyun Gong, Chris Hawblitzel, Shuvendu Lahiri, Jacob R. Lorch, Shuai Lu, Fan Yang, Ziqiao Zhou, and Shan Lu. 2025. AutoVerus: Automated Proof Generation for Rust Code. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2, Article 396 (2025).
- [46] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. 2023. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- [47] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *PLDI*.
- [48] Emily Yu, Armin Biere, and Keijo Heljanko. 2021. Progress in Certifying Hardware Model Checking Results. In *CAV*.